

Model-as-a-Service Architecture

A production-oriented architecture brief for specialised athlete-intelligence models, trusted historical trends, and efficient model context.

Design thesis
Facts are established by deterministic services. Models explain only the compact, authorised result.

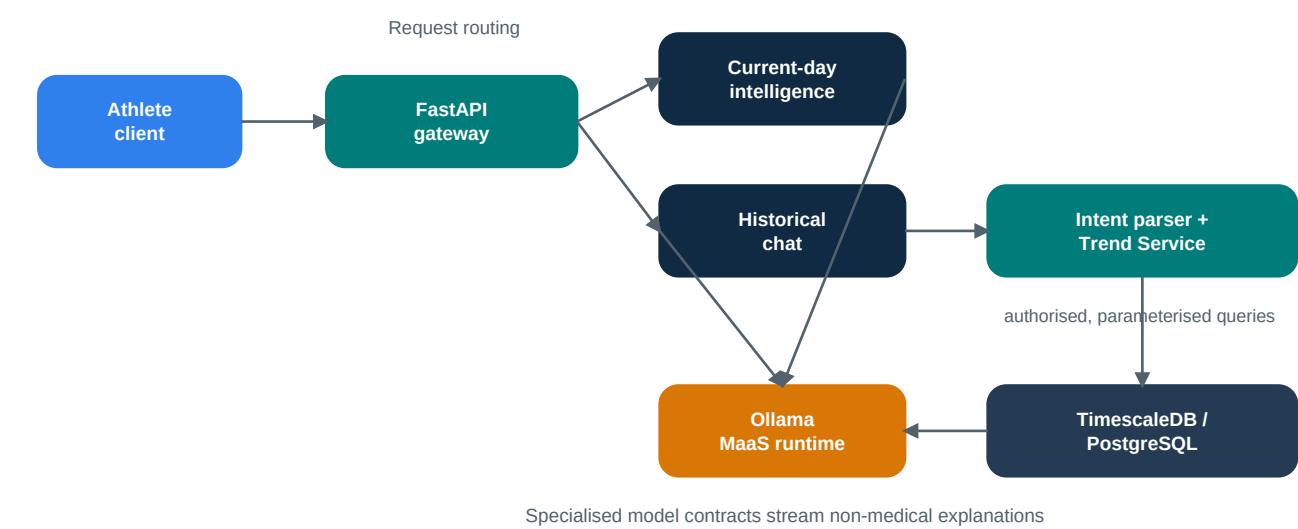
SYSTEM	RHYTHMX INTELLIGENCE API
PATTERN	Prompt Microservices Architecture / MaaS
MODEL RUNTIME	Ollama with specialised gpt-oss:20b contracts
PRIMARY USE	Non-medical recovery, readiness, injury-risk, and historical-trend explanations
STATUS	Architecture reference; not clinical-performance evidence

Decision-support only. The system does not diagnose, treat, interpret raw ECG waveforms, or guarantee an injury outcome.

Prepared from the current RhythmX codebase, architecture.md, README.md, prompts.md, and the detailed architecture design reference.

1. System architecture

RhythmX uses FastAPI as a single orchestration boundary. It routes a request either to a current-day specialist model or through deterministic historical retrieval before invoking the chat model. The Ollama runtime receives serialised context only; it does not receive database credentials or direct database access.



LAYER	RESPONSIBILITY	TRUST BOUNDARY
Client	Sends message, timezone, and upstream context.	Untrusted input
FastAPI gateway	Validates, routes, streams, and coordinates services.	Application boundary
Trend Service	Uses allowlisted queries and calculates summaries.	Trusted calculation
TimescaleDB	Stores Django-created daily model outputs.	Private data boundary
Ollama	Explains compact facts within prompt guardrails.	No database access

Implemented versus target: the current gateway validates a numeric athlete_id in the request body. Production should resolve an authorised athlete identity from a verified JWT or trusted upstream gateway before every health-data query.

2. MaaS service contracts

MaaS in RhythmX means treating a model capability as a narrow, versioned service contract. A Modelfile declares a model name, context window, generation defaults, and system instructions. The gateway maps each capability to a named Ollama tag and sends only task-relevant context.

CAPABILITY	MODEL TAG	ROLE	CONTEXT
Chat	rhythmx-chat:latest	Historical and general non-medical synthesis	8,192
Recovery	rhythmx-recovery:latest	Translate pre-computed recovery drivers	4,096
Readiness	rhythmx-readiness:latest	Explain capacity to tolerate current load	4,096
Injury risk	rhythmx-risk:latest	Explain strain/vulnerability without exposing probability	4,096
Overall state	rhythmx-overall:latest	Synthesize specialist outputs into a hero card	4,096

All current contracts use **gpt-oss:20b**, temperature 0.2, and top_p 1.0. This is logical model separation: in the default deployment, the models share one Ollama runtime and are not yet independent GPU pools or autoscaling units.

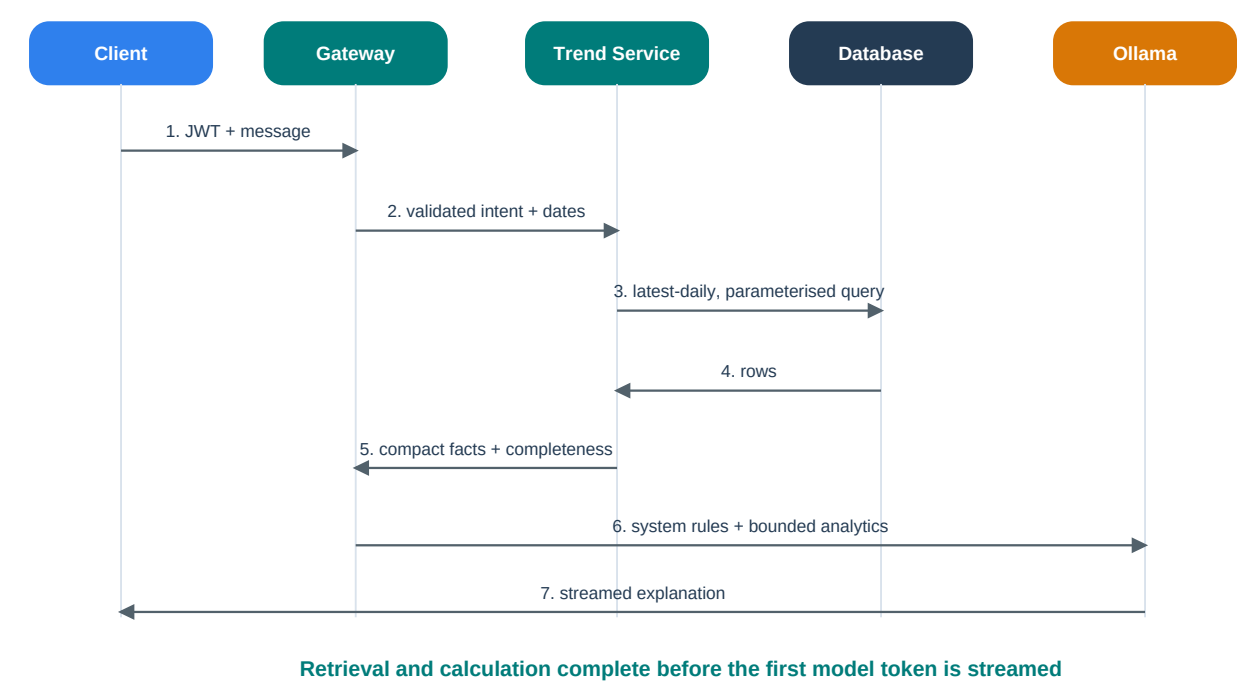
The specialised prompts are interpreters, not scoring engines. Upstream systems validate ECG-derived features, sleep, nutrition, workload, recovery, and wellness signals; they calculate scores and supply ranked drivers. The model converts those facts into constrained language.

Why it matters: a recovery request does not carry risk, historical-chat, or overall-state instructions. Prompt changes are isolated, reviewable, and testable instead of creating regressions in one monolithic instruction set.

Physical service separation is a later optimisation. Split chat, specialist, and routing workloads into independent GPU pools only after measured queueing, fault-isolation, and cost evidence justify the additional operational complexity.

3. Trend Service: trusted facts before language

Historical questions are answered by the trusted **Trend Service**, not by an LLM tool call. The service recognises supported metrics and periods, uses fixed table and column mappings, performs parameterised TimescaleDB/PostgreSQL queries, and calculates summary statistics before the chat model starts streaming.



STEP	CONTROL	OUTCOME
Intent and date resolution	Deterministic parser and athlete timezone	No LLM routing call for known phrases
Latest daily data	DISTINCT ON (date), ordered by newest time	One current record per day
Numerical aggregation	Python calculates average, range, change, direction, completeness	No model-side arithmetic or invented history
Parallel I/O	Primary metric and supporting signals fetched concurrently	Reduced database wait on the critical path
Prompt assembly	Compact ATHLETE_ANALYTICS_RESULT	The chat model receives facts, not rows or SQL

Trend Service currently supports recovery, readiness, injury risk, HRV, RHR, sleep, nutrition, hydration, ARI, SpO2, stress, soreness, and fatigue. It returns daily values, a summary, period comparison where requested, and data-quality coverage. Missing data is surfaced rather than calculated away.

4. Effective token utilisation and performance tuning

Effective token utilisation means spending model context on evidence and the final explanation, not on repeated instructions, raw wearable events, database rows, or model-side calculation. RhythmX applies token control at the prompt, retrieval, and output layers.

CONTEXT COMPONENT	PLANNING RANGE	CONTROL
System and safety instructions	500-800 tokens	One domain-specific Modelfile per endpoint
Athlete question	20-100 tokens	Request-size validation
Trend payload	300-800 tokens	Server-side filtering, aggregation, and period limits
Conversation memory	0 today; 200-600 target	Compact consented summary, never full-turn dump
Optional note retrieval	0 today; maximum 500 target	Permission-filtered, non-numerical supporting context
Final answer	150-300 target	Endpoint-specific output caps and concise format

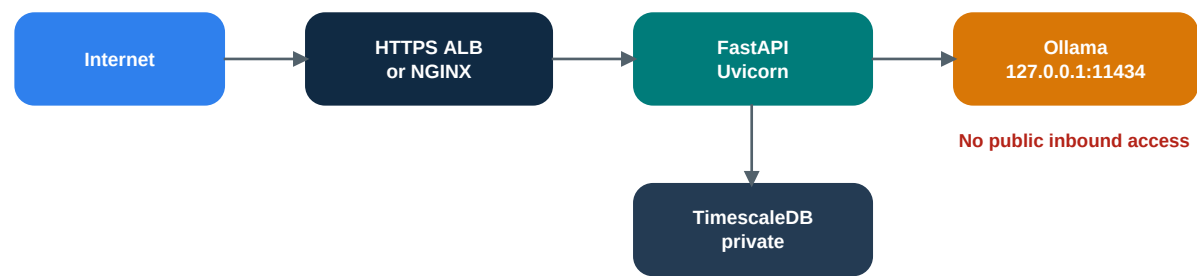
Current performance controls: deterministic intent parsing, async database pooling, concurrent primary/support retrieval, 4,096-token specialist contexts, an 8,192-token chat context, low reasoning effort, bounded thinking tokens, and a five-minute model keep-alive.

First tuning actions: make supporting metrics conditional by query type; set a maximum historical period and payload byte budget; align the specialist runtime output cap (currently up to 1,024) with its 512-token Modelfile default; measure first-token latency before increasing concurrency.

Do not claim a token reduction percentage until production telemetry compares the compact Trend Service payload with a controlled baseline using the same authorised data, model tag, and prompt version.

5. Private EC2 deployment and operational controls

The recommended deployment keeps inference private. FastAPI and Ollama can run on the same GPU EC2 host, with FastAPI calling Ollama through loopback. An HTTPS ALB or NGINX exposes the gateway - not the model runtime - to clients.



TLS terminates at the edge; only FastAPI reaches data and model services

AREA	RECOMMENDED CONTROL
GPU baseline	g6.xlarge with 24 GB VRAM for quantised inference; use g6e.xlarge with 48 GB only for demonstrated context or concurrency needs.
Network	Expose HTTPS at ALB/NGINX; restrict FastAPI port to the ALB security group; never expose Ollama port 11434 publicly.
Database	TLS, least-privilege read account, parameterised values, and per-athlete authorisation.
Secrets	Use a protected deployment environment or secrets manager for DATABASE_URL and credentials.
Health	Use /openapi.json for infrastructure liveness and /health/db for database readiness.
Runtime	Create the five model tags from checked-in Modelfiles; verify tags match MODEL_MAPPING before traffic.

Operational warning: current application logging and runtime dataset capture may retain health-adjacent request/response content. Production use requires consent-aware retention, redaction, encryption, access control, and an audit design that stores metadata rather than raw records.

6. Safety, measurement, and roadmap

Prompt controls complement - but cannot replace - authentication, data governance, deterministic calculations, schema validation, and human oversight. The system is designed to provide non-medical decision support only.

MEASURE	WHY IT MATTERS	DIMENSIONS
Time to first token / total latency	Detect retrieval or inference regression	endpoint, model tag, prompt size, status
Prompt / output tokens	Prove context minimisation and control cost	intent, period length, payload bytes
Trend data coverage	Prevent overconfident language	metric, requested vs available days
Safety and schema failures	Detect contract drift	prompt version, model tag, reason
Audit provenance	Reproduce a response safely	request ID, authorised athlete, date range, payload hash

Roadmap

- **Secure the boundary:** resolve athlete identity from verified authentication, enforce authorisation, protect secrets, and minimise sensitive logging.
- **Enforce contracts:** add Pydantic request models, JSON output schemas, prompt regression tests, and adversarial safety tests.
- **Measure and optimise:** collect token, latency, GPU, factuality, and data-quality metrics before changing model tiers or payload budgets.
- **Validate responsibly:** run a consented, representative evaluation of prediction horizon, calibration, recall, false alerts, and subgroup performance before external claims.

Decision rule: a reduction in tokens or latency is not a valid optimisation if it hides a material trend, data-quality limitation, or safety requirement.

Source basis: RhythmX architecture.md; README.md; prompts.md; docs/athlete_architecture.md; docs/rhythm_x_architecture_design.pdf; and the current FastAPI, Trend Service, and Modelfile implementation.

Fine-tuning and model release: governed MaaS adaptation

`train.py` adds an offline supervised fine-tuning capability. It is a candidate-model build step, not an automatic retraining or deployment mechanism. The existing Trend Service, Modelfiles, safety prompts, and release controls remain authoritative at inference time.



Rejected candidates return to data/configuration remediation - not to production inference

A training artifact becomes a MaaS model only after contract, safety, quality, and release gates pass

TRAINING COMPONENT	CURRENT CONFIGURATION IN train.py	ARCHITECTURAL INTERPRETATION
Base model	openai/gpt-oss-20b loaded in 4-bit mode	Candidate starts from the same model family used by MaaS contracts.
Sequence and adapters	2,048 max sequence; QLoRA rank 16, alpha 16, zero dropout	Memory-efficient adaptation; not proof of optimal quality or safety.
Target modules	Attention and MLP projection modules	Adapter scope is explicit and should be release-versioned.
Trainer	Batch size 2, accumulation 4, LR 2e-4, max_steps 100, seed 3407	Code defaults require reproducible experiment records and tuning evidence.
Export	rhythmx_finetuned_gguf with q4_k_m quantisation	A portable candidate artifact, not yet an Ollama tag or deployed endpoint.

Non-negotiable: Fine-tuning changes weights/adapters. Modelfiles still define the deployed task role, system safety rules, context budget, and response contract. A candidate must preserve or improve those contracts for each specialist endpoint.

Training data, evaluation, and promotion gates

The trainer expects ``dataset.jsonl`` examples with ``instruction`` and ``output``. Runtime interaction data has a different schema and can contain health-adjacent content, so it must not be used as a direct training source.

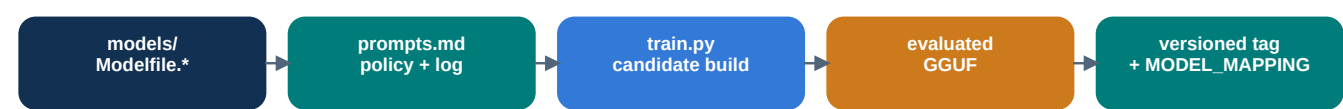
GATE	REQUIRED CONTROL	EVIDENCE BEFORE PROMOTION
Dataset lineage	Consent/authorisation, de-identification, quality filtering, immutable split and hash	Dataset card; training/validation/test manifest; owner approval.
Training reproducibility	Pinned training environment and recorded configuration	Base model, code revision, LoRA config, seed, artifact hash, quantisation.
MaaS contract	Specialist schema tests and historical chat grounding	JSON validity; disclaimer/data-limit behaviour; no SQL or model-side facts.
Safety	Adversarial and domain-specific checks	No diagnosis, raw ECG interpretation, probability exposure, or prompt-injection bypass.
Benchmark	Candidate-versus-baseline evaluation	Held-out quality, token, latency, GPU, and error-rate comparison.
Release	Versioned registration and rollback	Modelfile/tag, model card, approver, MODEL_MAPPING change, prior rollback tag.

A valid release registers the evaluated GGUF through a versioned Ollama Modelfile/tag, retains the relevant prompt contract, and updates MODEL_MAPPING only after approval. A rejected candidate remains offline. Training loss alone is never a release criterion.

Evidence status: the repository contains the QLoRA configuration and GGUF export path only. No approved training dataset, completed run, evaluation result, promoted fine-tuned model, or clinical prediction claim is recorded.

Release-policy revision: prompt and training governance

This revision consolidates prompt and fine-tuning governance into **prompts.md**. The duplicate **prompt.md** file has been removed. Canonical runtime prompts remain in **models/Modelfile.***; prompts.md records their contracts, refinements, training data policy, and release evidence.



Single source of truth for prompt governance: prompts.md

A candidate does not bypass the Modelfile contract, safety checks, or promotion gate

ARTIFACT	AUTHORITATIVE ROLE	RELEASE RULE
models/Modelfile.*	Deployed model role, safety instructions, context and generation defaults	Version with every model/prompt release.
prompts.md	Prompt ledger, fine-tuning policy, evaluation log, and promotion controls	Single documentation source for prompt and candidate-model governance.
train.py	Offline QLoRA configuration and GGUF export	Produces a candidate only; it cannot deploy or update MODEL_MAPPING.
GGUF candidate	Quantised fine-tuned artifact	Requires held-out quality, safety, schema, token/latency, and approval gates.
main.py / MODEL_MAPPING	Runtime tag selection	Update only through an approved, rollback-capable release.

Data boundary. `train.py` accepts reviewed `instruction`/`output` JSONL examples. Runtime logs have a different schema and may contain health-adjacent content; they require consent, de-identification, quality filtering, provenance, and immutable split validation before they are ever used for training.

Release boundary. Fine-tuning changes weights or adapters, while Modelfiles define the specialist MaaS contract. A model is rejected if it fails factual grounding, JSON/schema behaviour, medical-safety, prompt-injection, or data-limit disclosure checks - even if training loss improves.

Evidence status. The repository contains a QLoRA configuration and GGUF export path. It does not record an approved dataset, completed run, evaluated candidate, promoted tag, or clinical prediction result.